PROJECT REPORT

Accessibility Auditor

A tool that scans a web page for accessibility problems, gives it a score out of 100, and explains what to fix.

Project	Accessibility Auditor (WCAG checker)
Type	Web quality / testing tool — browser automation and reporting
Built by	Anu
Main tools	Python, Playwright, axe-core 4.14.0, Streamlit
Interfaces	Command line tool and a local web app
Outputs	Markdown, printable HTML, JSON
Automated tests	29 — passing, with continuous integration
Source code	github.com/anusanu162-sudo/accessibility-auditor
Date	November 2026

Python · Playwright · axe-core · Streamlit · pytest · GitHub Actions · MIT licensed

This report explains what the project does, how it works, how it was tested, what it cannot do, and the complete algorithm in plain English.
Every claim in this document can be checked against the public repository.

Contents

1. 1. Project at a Glance
2. 2. What This Project Does
3. 3. Why It Matters
4. 4. What Accessibility Means in Practice
5. 5. How the Scanner Works
6. 6. The Checks It Runs
7. 7. The 0–100 Score
8. 8. The Reports
9. 9. Real Scans and What They Show
10. 10. Inside the Project Folder
11. 11. How to Run It
12. 12. Engineering Quality: Tests and CI
13. 13. Strengths of the Project
14. 14. Limitations (Honest List)
15. 15. Future Improvements
16. 16. Glossary in Simple English
17. 17. The Complete Algorithm — Explained for HR

HOW TO READ THIS REPORT

Sections 1 to 15 explain the project for a technical reader. Section 16 explains every term in plain language. Section 17 is written for a non-technical reader, such as an HR interviewer, and contains the complete algorithm from beginning to end.

The short version: this is a working command-line tool and web app that opens a page in a real browser, runs an industry-standard accessibility engine against it, converts the findings into a score and a readable report, and ships with tests, continuous integration and documented limits.

1. Project at a Glance

The Accessibility Auditor checks a web page for common accessibility mistakes: images without alternative text, text that is too faint to read, form fields with no label, buttons with no name, broken heading order, and misuse of ARIA attributes. It reports each problem with the exact element that caused it, gives the page a score out of 100, and exports the result in three formats.

100	3	29	2
POINT SCORE	REPORT FORMATS	AUTOMATED TESTS	INTERFACES (CLI + WEB)

Question	Answer
What problem does it solve?	Accessibility problems are invisible to the person who built the page, but they block real users. Finding them by hand is slow.
Who is it for?	Developers, students, QA testers, and anyone who wants a quick accessibility check before publishing a page.
What goes in?	One public URL, for example `https://example.com`.
What comes out?	A score, a letter grade, a severity breakdown, and a list of failed rules with affected elements.
How is the page checked?	A real Chrome browser loads the page, then axe-core, an open-source accessibility engine, inspects the rendered result.
How long does a scan take?	Usually 3–8 seconds, depending on the page and network.
Is it deployed?	No. It runs locally. The code, tests and CI are public on GitHub.
What is the honest status?	A solid working tool and learning project, not a certification product. It finds a useful share of problems, not all of them.

> **IN SIMPLE WORDS**
>
> You give it a web address. It looks at the page the way a browser sees it, lists the accessibility mistakes it can detect, scores the page, and writes a report you can send to a developer.

2. What This Project Does

2.1 For the user

1. The user enters a URL in the command line or the web app.
2. The tool opens that page in a real browser, exactly as a visitor would see it.
3. It runs dozens of accessibility rules against the rendered page.
4. It groups the failures by severity and calculates a score out of 100.
5. It shows each failed rule with the CSS selector and HTML of every affected element.

6. The user downloads the result as Markdown, printable HTML or JSON.

2.2 Features built into the tool

Feature	What it gives the user
Command-line interface	One command, reports written to a folder. Easy to put in a script or CI pipeline.
Streamlit web app	Type a URL, press a button, see the score and expand each issue in the browser.
Severity filter	Hide minor issues and focus on critical and serious barriers first.
Element-level detail	Shows the CSS selector and a short HTML snippet, so the fix is easy to locate.
Three export formats	Markdown for GitHub, printable HTML for sharing, JSON for other tools.
`--fail-under` gate	Exit with code 1 when the score is too low, so a build can be blocked automatically.
Bot-protection guard	Detects CAPTCHA and "Just a moment" pages and refuses to score them instead of producing a meaningless result.
Vendored engine	axe-core is stored inside the project, so scans do not depend on a third-party CDN at run time.
Offline tests	The test suite needs no network and no browser, so it runs anywhere in seconds.

2.3 Behind the scenes, in one paragraph

The scanner uses Playwright to start a headless Chrome browser and open the URL. After the page settles, it injects the axe-core JavaScript file into the page and asks it to inspect the whole document. axe-core returns a structured list of failed rules. The Python code reduces that large response to the fields it needs, converts it into simple data objects, calculates the score, and passes the result to three report writers. Finally the browser is closed and the files are saved. Nothing is uploaded anywhere.

> **IN SIMPLE WORDS**
>
> It is a browser robot that opens a page, runs an accessibility checklist on it, and writes down what is wrong and where.

3. Why It Matters

The web is how people apply for jobs, pay bills, study, book appointments and reach government services. When a page is built without accessibility in mind, some people are simply locked out — not because they lack the ability, but because the page was not built for them.

The World Health Organization estimates that about **1.3 billion people**, roughly 16% of the world, live with significant disability. Add temporary conditions such as a broken wrist or a bright screen in sunlight, and the number of people who benefit from accessible design is far larger.

3.1 Common barriers and who they affect

Barrier	Who is blocked
Image with no alt text	A blind user hears “image” instead of the information the image carries.
Text with low contrast	People with low vision, colour blindness, or anyone using a phone in sunlight.
Form field with no label	A screen reader cannot say what the field is for, so the user has to guess.
Button with no name	The control is announced as just “button”. The user cannot tell what it does.
Heading levels out of order	Screen reader users navigating by headings lose the structure of the page.
Content unreachable by keyboard	People who cannot use a mouse cannot operate the page at all.
Zoom disabled	People who need larger text are prevented from enlarging it.

3.2 The legal side

- **WCAG 2.2** is the current Web Content Accessibility Guidelines recommendation from the W3C, published in 2023.
- **India:** the Rights of Persons with Disabilities Act, 2016 requires accessible public digital services.
- **European Union:** the European Accessibility Act applies to many digital products and services from June 2025.
- **United States:** the Americans with Disabilities Act has been applied to websites in many court cases.

Laws differ by country and by sector. This project does not give legal advice, and a good score is not a statement of legal compliance.

> **THE HONEST PROBLEM**
>
> Automated tools find only part of the problem. They are excellent at catching mechanical mistakes such as missing alt text, but they cannot judge whether the alt text makes sense, whether the reading order is logical, or whether a screen reader announces the page clearly. This project is a first filter, not a replacement for human testing.

4. What Accessibility Means in Practice

WCAG organises accessibility into four principles, often remembered as **POUR**. They are a useful way to understand why the rules in this project exist.

Principle	Plain meaning	Example problem this tool catches
Perceivable	The user must be able to sense the content.	An image with no text alternative; grey text on white with a contrast ratio below the minimum.
Operable	The user must be able to use the controls.	A clickable `<div>` that cannot be reached with the keyboard; zoom disabled in the viewport tag.
Understandable	The page must behave predictably and be readable.	A missing page language; headings that jump from level 1 to level 4.

Robust	The page must work with different browsers and assistive technology.	Invalid ARIA roles; ARIA roles placed inside the wrong parent element.

4.1 WCAG conformance levels are not the same as impact levels

WCAG defines three conformance levels: **A** (minimum), **AA** (the usual legal target) and **AAA** (enhanced). The tool does not claim a conformance level. Instead, each failed rule is labelled with axe-core's own urgency rating:

Impact	What it means here
Critical	A barrier that can stop some users completely, such as a control with no name.
Serious	A significant barrier that makes the page hard or confusing to use.
Moderate	A real problem that adds friction but usually leaves a workaround.
Minor	A small issue, often about polish or best practice.

> **WHY THIS DISTINCTION MATTERS**
>
> "Impact" answers "how badly does this hurt a user?". "Conformance" answers "does this page meet a specific WCAG success criterion?". They are related but not identical. This tool reports the first and never claims the second.

4.2 What a good result actually looks like

A page with no automated violations is a good sign, not a certificate. A responsible workflow is: fix what the tool finds, then test with a keyboard, then test with a screen reader, then ask a real user to try a task. This project automates the first step well and says so clearly.

5. How the Scanner Works

5.1 Why a real browser is needed

Many accessibility problems only exist after the browser has built the page. A modern site may render its menu with JavaScript, load fonts late, or set colours in a stylesheet. A simple HTML download would miss all of that. Playwright drives an actual Chrome or Edge browser, so the tool checks the page in the same state a user sees.

5.2 What axe-core is

axe-core is an open-source accessibility testing engine made by Deque Systems, licensed under the Mozilla Public License 2.0. It is the same engine used behind Lighthouse's accessibility audit and by many commercial testing tools. It contains dozens of rules covering images, forms, colour contrast, ARIA, document structure, tables and best practices.

Version **4.14.0** is stored inside the project at `a11y_auditor/vendor/axe.min.js` (about 637 KB). Keeping a local copy means a scan never fails because a content delivery network is down or blocked, and the exact rule set is reproducible.

5.3 The scan, step by step

1. Start a browser
Playwright launches installed Chrome, Edge, or bundled Chromium in headless mode

▼

2. Open the URL
Desktop viewport 1366 × 900 · configurable timeout · waits for the load event

▼

3. Let the page settle
Extra wait (1.2 s by default) for late JavaScript, fonts and lazy content

▼

4. Check for a bot wall
If the title is a CAPTCHA or "Just a moment" page, stop and explain instead of scoring it

▼

5. Inject axe-core
Add a script tag; if a strict CSP blocks it, evaluate the source directly as a fallback

▼

6. Run the rules
axe.run(document) returns violations, passes and incomplete checks

▼

7. Reduce the result
Keep only the fields needed: rule id, impact, help text, tags and affected elements

▼

8. Score and report
Calculate the score, render Markdown / HTML / JSON, close the browser

5.4 Robustness decisions

Decision	Reason
Try the requested browser, then fall back	If Chrome is not installed, the tool still works with Playwright's own Chromium, with a clear error if neither exists.
Script-tag injection with an eval fallback	Some sites use a Content Security Policy that blocks added script files.
Refuse to score bot-protection pages	A CAPTCHA page is not the site. Scoring it would produce a false result.
Cap stored elements per rule	A page with 200 broken images should not send a giant result across the browser boundary or flood the report.
Close the browser in a `finally` block	Processes are cleaned up even when a scan fails.

> **IN SIMPLE WORDS**
>
> The tool does not guess from the HTML source. It opens the page in a real browser, lets it finish loading, then runs a professional accessibility checklist inside the page.

6. The Checks It Runs

The tool runs every rule that axe-core enables by default, including WCAG 2.0, 2.1 and 2.2 rules and best-practice rules. The table below groups the most important ones by what a user experiences.

Area	Example rule	What it catches
Images	`image-alt`	An informative image with no `alt` text, or a decorative image that is not marked as decorative.
Colour	`color-contrast`	Text whose contrast ratio against its background is too low for normal reading.
Forms	`label`	An input with no programmatic label, so a screen reader cannot announce its purpose.
Control names	`button-name`, `link-name`	A button or link with no readable text or accessible name.
Structure	`heading-order`, `region`	Headings that skip levels; page content that sits outside any landmark region.
ARIA	`aria-required-parent`	An ARIA role used in a place where it cannot work, such as a tree item outside a tree.
Keyboard	`scrollable-region-focusable`	A scrollable area that cannot be reached or scrolled with the keyboard.
Document basics	`html-has-lang`, `frame-title`	A page with no declared language; an iframe with no title.
Tables	`td-headers-attr`	Data tables whose header cells are not connected to the data cells correctly.
Zoom and viewport	`meta-viewport`	A page that disables pinch zoom or sets a maximum scale.

6.1 What the tool reports for each failure

- **Rule id and title** — for example `image-alt`, “Images must have alternative text”.
- **Impact** — critical, serious, moderate or minor.
- **Description and documentation link** — so the developer can read the full rule.
- **WCAG tags** — which success criteria the rule maps to, when applicable.
- **Affected elements** — a CSS selector and a short HTML snippet for each one.
- **Failure summary** — axe-core’s short explanation of what to change.

6.2 What it does not report

A rule that passes is counted but not listed in detail. Rules marked “incomplete” by axe-core are counted separately, because the engine could not decide automatically. The report shows that count so the user knows a human should look.

> **IMPORTANT**
>
> Automated rules check what can be expressed as a machine rule. They cannot tell whether an alt text is *useful*, whether the tab order is *logical*, or whether an error message is *understandable*. Those need a person.

7. The 0–100 Score

The score is a summary, not a measurement of WCAG compliance. It exists so a team can see progress and compare pages. The formula is deliberately simple enough to explain to a non-technical person.

7.1 Step one: what each failed rule costs

Impact	Points per rule	Most it can cost in total
Critical	10	50
Serious	6	30
Moderate	3	15
Minor	1	5

7.2 Step two: count affected elements, but stop at three

Each rule’s cost is multiplied by the number of affected elements, but never by more than three. A page with 200 images missing alt text therefore costs the same as a page with three. Without this limit, one repeated mistake would hide every other problem on the page.

7.3 Step three: cap each severity

Even a page with 30 critical rules should not automatically score zero, because zero hides everything the page gets right. Each severity can only take away the maximum shown in the table above. The four caps add up to 100, so a score of 0 is still possible when every category is saturated.

7.4 Step four: subtract and grade

`score = max(0, 100 - total deduction)`. The letter grade follows:

Grade	Score	Meaning used by the tool
A	90–100	Good — only minor problems found.
B	80–89	Fair — several issues should be fixed.
C	70–79	Needs work — real barriers for some users.
D	60–69	Poor — many users will struggle.
F	0–59	Failing — serious barriers across the page.

7.5 A complete worked example

The demo page in `examples/bad-page.html` fails nine rules. Here is the exact calculation, using the real scan output:

Rule	Impact	Elements	Counted	Points
`button-name`	Critical	1	1	10
`image-alt`	Critical	1	1	10
`color-contrast`	Serious	1	1	6
`html-has-lang`	Serious	1	1	6
`link-name`	Serious	1	1	6
`heading-order`	Moderate	1	1	3
`landmark-one-main`	Moderate	1	1	3
`meta-viewport`	Moderate	1	1	3
`region`	Moderate	10	3	9

SEVERITY TOTALS BEFORE CAPS

Critical 20 · Serious 18 · Moderate 18 · Minor 0

AFTER CAPS

Critical 20 (cap 50) · Serious 18 (cap 30) · Moderate 15 (cap 15) · Minor 0

47 OUT OF 100 — GRADE F

100 − (20 + 18 + 15 + 0) = 47

7.6 Why the caps exist

The first version of the score had no severity caps. When tested against `python.org`, it returned **0/100** from seven failed rules. That number was technically consistent but useless: it told the reader nothing about which problems were worse, and it made any improvement invisible. The caps were added after that test, and the same page then scored **17/100**. Still poor, but informative.

The caps are a product judgement, not an accessibility standard. The report and README both say so. The grade is a progress indicator, not a compliance verdict.

> **IN SIMPLE WORDS**
>
> Start at 100. Take away points for each kind of mistake, with worse mistakes costing more. Stop counting repeats after three examples of the same mistake, and stop one category from wiping out the whole score. Then turn the number into a letter.

8. The Reports

A scan produces up to three files, all from the same result object. They contain the same facts, formatted for different readers.

Format	Best for	What it contains
Markdown	GitHub issues, pull requests, README notes	Summary, severity table, one section per violation, element selectors and HTML snippets.
HTML	Sharing with a client or printing to PDF	A self-contained page with styling, score cards, severity table and every violation. No internet needed.
JSON	Other tools, dashboards, automation	Structured data: URL, title, timestamp, score, grade, counts, and a full array of violations and elements.

8.1 What every report includes

- The page URL and title, plus the scan date and duration.
- The score, grade and a one-line verdict.
- Counts of failed rules, affected elements, passed rules and checks needing review.
- A severity breakdown: critical, serious, moderate, minor.
- Every violation sorted from worst to least severe, then by number of affected elements.
- For each violation: the rule, its impact, a description, a documentation link, WCAG tags, and each affected element.
- The honest footer: automated checks catch only part of the problem.

8.2 Printable HTML instead of a separate PDF library

The HTML report is designed to print cleanly on A4. Instead of adding a heavy PDF dependency, the project tells the user to open the HTML file and choose **Print → Save as PDF**. That keeps the dependency list small and the output predictable. The repository includes a real sample at `docs/sample-report.html`.

> **DESIGN CHOICE**
>
> The reports never say “this page is WCAG compliant”. They say what was checked, what failed, and what a human should still verify. That restraint is deliberate.

> **IN SIMPLE WORDS**
>
> One scan gives you a GitHub-friendly text report, a polished page you can print or send, and a data file that other software can read.

9. Real Scans and What They Show

These are real results from the finished tool. They are included because a portfolio project should show evidence, not only claims.

Page	Score	Grade	Rules failed	What was found

`examples/bad-page.html` (deliberately broken)	47	F	9 (18 elements)	Missing button name, missing alt text, low contrast, no page language, empty link, heading jump, missing landmarks, zoom disabled, content outside landmarks.
`example.com`	85	B	3 (9 elements)	Only best-practice structure issues: content not wrapped in landmarks. No critical or serious failures.
`python.org`	17	F	7 (43 elements)	Two critical and four serious rules, including ARIA tree items outside a tree, plus contrast and hidden-navigation issues.

9.1 Reading the results honestly

A low score does not mean a site is unusable. The python.org result is driven by its hidden navigation: items carry the ARIA role `treeitem` without a surrounding tree, and the same kind of issue repeats many times. That is a genuine bug worth fixing, but most sighted visitors never notice it. The report lists the exact elements so a developer can judge.

Equally, a high score is not proof of quality. `example.com` is a deliberately tiny page. It cannot demonstrate good form design, keyboard flow or screen reader behaviour because it has none of those things.

9.2 A scan that refused to produce a result

The W3C's intentionally bad demo page was an obvious test target. When scanned, it returned a Cloudflare page titled "Just a moment…" instead of the real content. The first version of the tool happily reported "1 critical issue, score 90" against the CAPTCHA screen. That was a false result.

The fix was a guard: if the page title matches a known bot-protection screen, the scan stops with a clear message and exit code 2. It is a small feature, but it shows the difference between producing a number and producing a trustworthy number.

> **IN SIMPLE WORDS**
>
> The tool was tested on a broken demo page, a tiny clean page and a large real website. It also learned to refuse when a website blocks robots, instead of inventing a score.

10. Inside the Project Folder

The code is split by responsibility. Each file has one job, which keeps the scanner, the scoring logic, the reports and the interfaces independent of each other.

File	What it does
`a11y_auditor/models.py`	Defines the three data structures: one affected element, one failed rule, one complete result.
`a11y_auditor/auditor.py`	Starts the browser, loads the page, injects axe-core, runs the rules and returns an `AuditResult`.

`a11y_auditor/scoring.py`	Converts violations into a score, grade and short verdict using the documented formula.
`a11y_auditor/report.py`	Renders the same result as Markdown, standalone HTML or JSON.
`a11y_auditor/cli.py`	The command-line interface, file naming, format validation and the `--fail-under` exit code.
`a11y_auditor/vendor/axe.min.js`	The vendored axe-core 4.14.0 engine, licensed MPL-2.0.
`app.py`	The Streamlit web interface: URL input, scan button, metrics, filters, expandable issues and download buttons.
`tests/test_scoring.py`	Tests the weights, both caps, grade bands, impact counting and verdicts.
`tests/test_report.py`	Tests JSON shape, sorting, Markdown content, HTML escaping and standalone structure.
`tests/test_cli.py`	Tests report file names, the dot-in-host regression, and validation before a scan.
`tests/conftest.py`	Makes the project importable when pytest runs from any directory.
`examples/bad-page.html`	A small page with deliberate mistakes, used for demos and manual checks.
`docs/sample-report.html`	A real report generated from the demo page, committed as evidence.
`pyproject.toml`	Package metadata, dependencies and the `a11y-auditor` console command.
`requirements.txt`	The short list of runtime and test packages.
`pytest.ini`	Tells pytest where the tests live.
`.github/workflows/ci.yml`	GitHub Actions workflow: install dependencies and run the tests on every push and pull request.
`README.md`	What the project does, how to install it, how to use it, the scoring model, credits and honest limits.
`LICENSE`	MIT for the project code, with a clear note that axe-core keeps its own MPL-2.0 licence.
`.gitignore`	Keeps the virtual environment, caches and generated reports out of Git.

> **IN SIMPLE WORDS**
>
> One file opens the page, one file calculates the score, one file writes the report, one file is the command line, one file is the web page. The tests check the maths and the reports without needing the internet.

11. How to Run It

11.1 What you need first

- Python 3.10 or newer.
- Google Chrome or Microsoft Edge installed, or Playwright's own Chromium.

- Internet access for the page you want to scan (the tool itself runs locally).

11.2 Setup

1. Open a terminal in the project folder.
2. Create a virtual environment: `python -m venv venv`
3. Activate it: `venv\Scripts\activate` on Windows, or `source venv/bin/activate` on macOS / Linux.
4. Install the packages: `pip install -r requirements.txt`
5. Optional: install the package itself with `pip install -e .` to get the short `a11y-auditor` command.

If Chrome and Edge are missing, run `playwright install chromium` and pass `--browser ""` to use the bundled browser.

11.3 Scan from the command line

Basic use:

```
python -m a11y_auditor.cli https://example.com
```

Useful options:

Option	What it does
`--out DIR`	Where to write the reports. Default: `reports`.
`--format md,html,json`	Choose any combination of formats.
`--browser chrome`	Choose `chrome`, `msedge`, or an empty string for bundled Chromium.
`--timeout 30000`	Page load timeout in milliseconds.
`--wait 1200`	Extra settle time after load, for late JavaScript.
`--fail-under 90`	Exit with code 1 if the score is below 90. Useful in CI.
`--quiet`	Print only the summary, not every violation.

11.4 Scan from the web app

```
streamlit run app.py
```

The app opens at `http://localhost:8501`. Enter a URL, press **Scan page**, then filter by severity and download any report format.

11.5 What a run looks like

```
Bed & Breakfast - demo page with accessibility problems
  score 47/100 (grade F)
  9 rules failed, 18 elements affected
  critical 2 | serious 3 | moderate 4 | minor 0

  1. [critical] button-name - Buttons must have discernible text (1 element(s))
       button
  2. [serious] color-contrast - Elements must meet minimum color contrast ratio thresholds (1
element(s))
```

```
        .faint
  ...

Reports written:
  reports/bad-page.md
  reports/bad-page.html
  reports/bad-page.json
```

> **IN SIMPLE WORDS**
>
> Install once, activate the environment, then either run one command or open the web page. The result is a readable report in a folder, plus a summary in the terminal.

12. Engineering Quality: Tests and CI

A working demo is not the same as a maintained project. Two things separate this project from a script that happens to run: the tests and the continuous integration pipeline.

12.1 The 29 tests

Test file	Tests	What is verified
`test_scoring.py`	12	Perfect page scores 100; each impact costs the right points; the three-element cap works; severity caps work; all categories saturated reaches 0; unknown impact falls back safely; grade boundaries; counts use rules not elements; every impact key always exists.
`test_report.py`	9	JSON has the expected fields and correct counts; violations are sorted worst-first; a clean page scores 100; JSON round-trips; Markdown contains the key sections; empty results are worded correctly; HTML escapes page content; HTML is a standalone document.
`test_cli.py`	8	File names keep dots in host names; schemes and trailing slashes are handled; file URLs use the file name; report paths are correct; long URLs are shortened; empty slugs fall back; bad formats are rejected before any scan.

The tests use synthetic data only. They need no network, no browser and no axe-core run, so the suite finishes in under a second and can run anywhere. That is a deliberate trade-off: the logic is tested thoroughly, while the browser scan is verified manually against real pages.

12.2 Continuous integration

The GitHub Actions workflow runs on every push to `main` and on every pull request. It installs Python 3.11, installs the dependencies, and runs `pytest`. A green badge on the README is a live signal that the tests still pass, not a screenshot pasted into the document.

12.3 Two real bugs the testing caught

Bug	What happened	Fix
Report names were truncated	`Path.with_suffix(".md")` treated the dot inside `www.w3.org` as a file suffix, producing `www.w3.md`.	Build the filename by hand and add a regression test for the exact case.

Tests passed locally, failed on CI	`conftest.py` added the `tests` folder instead of the project root. Locally, `python -m pytest` hid the mistake by adding the working directory. On CI, imports failed.	Point the path at the project root, then repeat the test with the plain `pytest` command that CI uses.

HONEST GAP

There is no end-to-end test that launches a browser and scans a page in CI yet. The browser scan is checked manually. Adding a local fixture page to GitHub Actions is the next testing step.

IN SIMPLE WORDS

The maths and the reports are checked automatically every time the code changes. The browser part is checked by hand. The tests already caught two real mistakes, including one that only appeared on a different computer.

13. Strengths of the Project

Strength	Why it matters
Uses a professional engine	axe-core is the same engine behind Lighthouse, not a homemade approximation of accessibility rules.
Checks the rendered page	Real browser rendering means JavaScript content and computed colours are included.
Two usable interfaces	The command line suits automation; the web app suits a person who wants to look around.
Actionable output	Every issue points to the exact element, not just a rule name.
Three export formats	The same result fits a GitHub issue, a client document or another program.
Documented scoring	The weights, caps and grades are written down and covered by tests, so the number is reproducible.
Automated tests and CI	Shows engineering discipline, not only a working demo.
Honest limits	The README and reports state what automation cannot catch and avoid compliance claims.
Handles real-world friction	Browser fallback, CSP fallback, bot-protection guard and timeouts are dealt with explicitly.
Packaging	A `pyproject.toml` and console entry point make it installable, not just a folder of scripts.
Licence hygiene	The project is MIT, and the bundled axe-core keeps its MPL-2.0 notice.
Calibrated after evidence	The scoring caps were changed because a real test produced a useless zero, and the change is documented.

> **IN SIMPLE WORDS**
>
> It is not just “a script that scans a website”. It is packaged, tested, documented, honest about its limits, and built on a trusted engine.

14. Limitations (Honest List)

Every tool has limits. Writing them down is part of building trust. These are the real limits of this version, not hypothetical ones.

Limitation	What it means in practice
Automation finds only part of the problem	Common estimates say automated checks catch roughly 30–50% of real accessibility issues. The rest needs human judgement.
Only the loaded state is checked	Content behind a login, a click, a menu, a modal or infinite scroll is not reached.
No keyboard or screen reader testing	The tool cannot judge tab order, focus traps, announcements or reading flow. Those still need a person.
The score is a heuristic	The weights and caps are product decisions, not a WCAG formula. A high score is not a compliance certificate.
One page per scan	No crawling, no sitemap, no site-wide summary. Each URL is a separate scan.
Content inside iframes is missed	axe-core runs on the main document. Frames would need separate injection and are not covered yet.
No screenshots	Issues are described with selectors and HTML snippets, not pictures of the problem.
No login or session support	Private pages behind authentication cannot be scanned.
Some sites block automation	Bot protection produces a refusal to score. That is safer than a false result, but it means the site cannot be checked.
Timing can change the result	JavaScript-heavy pages may still be loading. The extra wait is configurable, but it is not a guarantee.
Rule set is pinned	axe-core 4.14.0 is fixed in the repository. New rules and fixes arrive only when the vendored file is updated.
No browser test in CI	The scan path is verified manually; the automated suite covers scoring, reports and file naming.
Not deployed	Both interfaces run locally. There is no hosted demo or public API.
No PDF writer	PDF is produced by printing the HTML report, not by a dedicated PDF library.
No legal or compliance claim	The tool cannot certify WCAG conformance and does not try to.

> **IN SIMPLE WORDS**
>
> It is a strong first filter, not an accessibility expert. It catches the mechanical mistakes quickly and tells you where they are. A person still has to use the page with a keyboard and a screen reader to find the rest.

15. Future Improvements

15.1 Highest impact first

1. **End-to-end test with a local page.** Add a fixture page to the repository, install Chromium in CI, and assert the exact violations and score. This closes the largest testing gap.
2. **Crawl mode.** Accept a start URL, follow internal links up to a depth and page limit, and produce a site-wide summary.
3. **Screenshots per issue.** Use Playwright to capture the affected element and embed the image in the HTML report. A picture is faster to understand than a selector.
4. **Baseline comparison.** Save a scan, then compare the next one and show what was fixed, what is new and what remains.

15.2 Next

5. **SARIF export.** A format GitHub code scanning understands, so issues appear in the security tab automatically.
6. **Ready-made GitHub Action.** Let other repositories run the auditor with a few lines of YAML.
7. **Interactive states.** Allow a small script of clicks before the scan, so open menus and dialogs are checked too.
8. **Iframe support.** Inject axe-core into each frame and merge the results.
9. **WCAG level filter.** Choose A, AA or AAA rules only, for teams targeting a specific standard.
10. **Hosted demo.** Deploy the Streamlit app with a container, plus a queue and timeout so public use is safe.
11. **Localised reports.** Add Tamil and Hindi report text for local government and college websites.

> **IN SIMPLE WORDS**
>
> First make the testing complete. Then check a whole website instead of one page, show pictures of the problems, and make it easy to see whether a fix actually improved the score.

16. Glossary in Simple English

Term	Simple meaning
Accessibility	Building something so that people with disabilities can use it too.
WCAG	Web Content Accessibility Guidelines — the international rulebook for accessible websites.
WCAG 2.2	The current version of that rulebook, published by the W3C in 2023.
POUR	Four principles: Perceivable, Operable, Understandable, Robust.
Conformance level	A, AA or AAA — how fully a page meets WCAG. AA is the usual legal target.

Impact level	The tool's urgency label for a failed rule: critical, serious, moderate or minor.
axe-core	An open-source accessibility testing engine made by Deque Systems.
Rule	One specific accessibility check, such as "every image needs alt text".
Violation	A rule that the page failed.
Element / node	One piece of the page, such as an image, a button or a paragraph.
CSS selector	A short pattern that identifies an element, such as `img.hero`.
Alt text	A written description of an image for people who cannot see it.
ARIA	Extra HTML attributes that tell assistive technology what a custom control is and does.
Contrast ratio	How different two colours are in brightness. Low contrast makes text hard to read.
Screen reader	Software that reads a page aloud for a blind or low-vision user.
Keyboard navigation	Using Tab, Enter, Space and arrow keys instead of a mouse.
Landmark	A labelled page region such as header, navigation, main or footer, used for quick navigation.
Heading order	The sequence of heading levels. It should not jump from level 1 to level 4.
DOM	The browser's live tree of elements after the page has been built.
Headless browser	A real browser running without a visible window, controlled by code.
Playwright	The Python library that drives Chrome, Edge or Chromium from code.
CSP	Content Security Policy — a site setting that can block injected scripts.
Heuristic	A practical rule of thumb, not a guaranteed exact measurement.
CI (continuous integration)	A service that runs the tests automatically whenever code is pushed.
Regression test	A test written to stop an old bug from coming back.
Markdown	Plain text with simple formatting symbols, used on GitHub.
JSON	A structured text format that software can read easily.
SARIF	A standard file format for reporting code-analysis results to tools such as GitHub.
Virtual environment	A private folder of Python packages for one project, so projects do not interfere.

17. The Complete Algorithm — Explained for HR

This final section is written for a non-technical reader. No prior knowledge is assumed. Every step is one action. Nothing is skipped.

17.1 The one-minute explanation

> "I built a tool that checks whether a website is usable by people with disabilities. You give it a web address, and it opens the page in a real browser, the way a visitor would see it. It then runs a professional accessibility checklist and finds problems such as images with no description, text that is too faint to read, or buttons with no name. It groups the problems by how serious they are, gives the page a score out of 100, and writes a report that shows the exact element that needs fixing. It works from the command line and from a simple web page. I also wrote tests and set up automatic testing on GitHub, and I documented clearly what the tool cannot catch."

17.2 Algorithm A — How the system was built (from zero)

Step	Action	What it means
1	Understand the problem	Learn what makes a website hard to use for people with disabilities, and why manual checking does not scale.
2	Choose the scope	Check one page at a time, report what can be detected automatically, and do not pretend to replace human testing.
3	Choose an engine	Use axe-core, a trusted open-source accessibility engine, instead of inventing rules from scratch.
4	Make it reliable	Store axe-core inside the project so a scan does not depend on an external website.
5	Choose a browser driver	Use Playwright so the page is rendered by real Chrome, including JavaScript content.
6	Design the data	Define three simple objects: one affected element, one failed rule, one complete result.
7	Build the scanner	Open the page, wait for it to settle, inject axe-core, run the rules, and convert the result into those objects.
8	Handle real-world problems	Add a script fallback for strict security policies, a browser fallback, timeouts, and a guard against CAPTCHA pages.
9	Design the score	Give points per severity, count repeats up to three, cap each severity, subtract from 100, and map to a letter grade.
10	Build the reports	Write one result as Markdown, printable HTML and JSON from a single shared function.
11	Build the command line	Accept a URL, folder, formats, browser and timeout; validate the input; write the files; support a failure threshold.
12	Build the web app	Add a URL box, a scan button, score cards, a severity filter, expandable issue details and download buttons.
13	Write tests	Check the scoring maths, the report content, the HTML safety and the file naming, using synthetic data.
14	Add continuous integration	Run the tests automatically on every push and pull request, and show a green badge.

15	Test on real pages	Scan a deliberately broken demo page, a tiny clean page and a large real website; fix what the results reveal.
16	Document and publish	Write the README, licence, package metadata and honest limitations, then release the code on GitHub.

17.3 Algorithm B — What happens when a user scans a URL

Step	Action	What it means
1	User enters a URL	The command line or web app receives the address.
2	Check the input	The app rejects an empty address or one without `http://` or `https://`.
3	Start a browser	Playwright launches Chrome, Edge or bundled Chromium in the background.
4	Open the page	The browser loads the address and waits for the page load event, up to the timeout.
5	Wait a moment	Extra time lets late JavaScript, fonts and images finish appearing.
6	Check the title	If the page is a bot check or CAPTCHA, stop with a clear message rather than scoring it.
7	Load the checklist	Insert axe-core into the page; if the site blocks it, run the source directly as a fallback.
8	Run the rules	axe-core inspects the rendered document and returns every failed rule and affected element.
9	Reduce the result	Keep only the useful fields: rule, impact, explanation, tags and element details.
10	Calculate the score	Apply the weights, the three-element limit and the severity caps; produce the score and grade.
11	Build the reports	Render Markdown, HTML and JSON from the same result.
12	Show and save	The terminal prints a summary and file paths; the web app shows metrics and download buttons.
13	Close the browser	The browser is always closed, even if an error occurred.

17.4 Algorithm C — How the score is calculated (step by step)

Step	Action	What it means
1	Start with 100 points	The page is assumed perfect until a rule proves otherwise.
2	Take one failed rule	Look at its impact label: critical, serious, moderate or minor.
3	Find its weight	Critical is 10, serious is 6, moderate is 3, minor is 1.
4	Count affected elements	Count how many elements break that rule, but never count more than three.

5	Multiply	Weight × counted elements = points for that rule.
6	Add to its severity bucket	Keep a separate running total for critical, serious, moderate and minor.
7	Repeat for every rule	Every failed rule contributes to its own bucket.
8	Apply the severity caps	Critical can take at most 50, serious 30, moderate 15, minor 5.
9	Add the capped buckets	This is the total number of points removed.
10	Subtract from 100	Never go below zero. The result is the score.
11	Assign a grade	90+ is A, 80+ is B, 70+ is C, 60+ is D, below 60 is F.
12	Write a verdict	One plain sentence explains what the score means.

17.5 How to say it in an interview (five points)

1. **The problem:** many websites lock out people with disabilities, and the people who build them usually cannot see the problem.
2. **The approach:** use a trusted accessibility engine inside a real browser, then turn its output into a score and an actionable report.
3. **The engineering:** separate scanning, scoring, reporting and interfaces; package it; test the maths and reports; run the tests automatically on GitHub.
4. **The product:** give both a command line for automation and a web page for people, with severity filters and three export formats.
5. **The honesty:** state that automation catches only part of the problem, document every limitation, and never claim WCAG certification.

> **IF ASKED TO EXPLAIN IT IN ONE SENTENCE**
>
> “It is a quality-check tool for websites: it opens a page like a real visitor, runs a professional accessibility checklist, and tells you what is broken, how serious it is and where to fix it — while being clear about what still needs a human.”

17.6 Summary of the whole pipeline

Input
One public URL from the command line or web app

▼

Render
Real browser opens the page · waits for JavaScript · guards against bot walls

▼

Check
axe-core 4.14.0 runs dozens of accessibility rules on the rendered page

▼

Score
Weights 10 / 6 / 3 / 1 · three-element limit · severity caps · letter grade

▼

Report
Markdown · printable HTML · JSON · every issue with selector and snippet

▼

Ship
29 tests · GitHub Actions · README with honest limits · MIT licence

---